

# **PROJECT REPORT STUDENT RECORD MANAGEMENT SYSTEM (SRMS)**



**DONE BY**

**NAME: T HARI VENKAT**

**REGISTRATION NUMBER: AP24110011941**

**CSE 2<sup>ND</sup> YEAR THIRD SEMESTER**

**SECTION-W**

**SUBMITTED TO**

**RAKESH RAMA RAJU**

**CODING SKILLS – I (CSE 201)**

## ABSTRACT

This project presents a Student Management System designed to simplify the process of handling student information using the C programming language. The system provides a digital platform where student records can be safely stored, accessed, and managed without the need for manual paperwork. It incorporates a secure login module with role-based access, allowing administrators full control over all operations while granting limited access to regular users.

The application uses structures for organizing student data and file handling to store records permanently, ensuring that information remains intact even after the program is closed. Core functions such as adding, updating, searching, displaying, and deleting records are implemented through a clean, menu-driven interface that improves usability. By integrating modular programming techniques, the system enhances clarity, reduces errors, and makes maintenance easier. This project demonstrates how fundamental C programming concepts can be applied to create an efficient, reliable, and practical student information management tool.

## Introduction

**The Student Management System is a** comprehensive console-based application developed using the C programming language, designed to efficiently organize, store, and manage student information. The system provides a structured and user-friendly environment where both Admin and User roles can access various functionalities based on their permissions. The entire system is divided into multiple smaller functions such as `addStudent()`, `searchStudent()`, `updateStudent()`, etc.

Overall, the Student Management System is an excellent example of applying C programming concepts like structures, conditional statements, loops, file I/O, string handling, and function modularity to create a fully functional, real-time application. It demonstrates how fundamental programming knowledge can be combined to develop a practical system for educational institutions to manage student data effectively.

## Objectives

To develop a secure Student Management System using C programming that allows safe and controlled access through file-based login authentication.

- To organize student information efficiently by using structures (struct) for storing details like ID, name, department, marks, and contact information in a systematic format.
- To implement complete data-handling operations (CRUD) including:
  - Adding new student records
  - Displaying all stored records
  - Searching for students
  - Updating existing information
  - Deleting outdated records
- To apply role-based access control, giving Admins full privileges while restricting normal Users to limited operations for safety and transparency.

- To ensure long-term data storage through file handling so that all student records remain available even after the program is closed.

## Software Requirements

**Operating System:** Windows 7/8/10/11 (or any OS that supports C Compiler).

**Programming Language:** C programming.

**Compiler:** GCC / MinGW / Turbo C / Visual Studio Code C compiler.

**Editor / IDE:** CodeBlocks, Dev-C++, VS Code, or any text editor that supports C.

**File System:** Supports text file creation for storing student records and login data.

**Memory:** Minimum 2GB RAM (for smooth program execution).

**Storage:** Few MB of disk space for saving .txt files.

**Additional Tools:** Standard C libraries (stdio.h, string.h, stdlib.h).

# System Design

## **Credential Setup:**

The program first prompts the user to create a username and password for secure access.

## **Login Authentication:**

The user enters their login credentials. The system verifies them through file-based authentication.

## **Role Identification:**

After successful login, the system checks the user's role (Admin or User) and loads the appropriate menu.

## **Admin Module:**

Admin has full control and can perform all functions:

- Add new student records
- Display all student details
- Search for a student
- Update existing data
- Delete student records

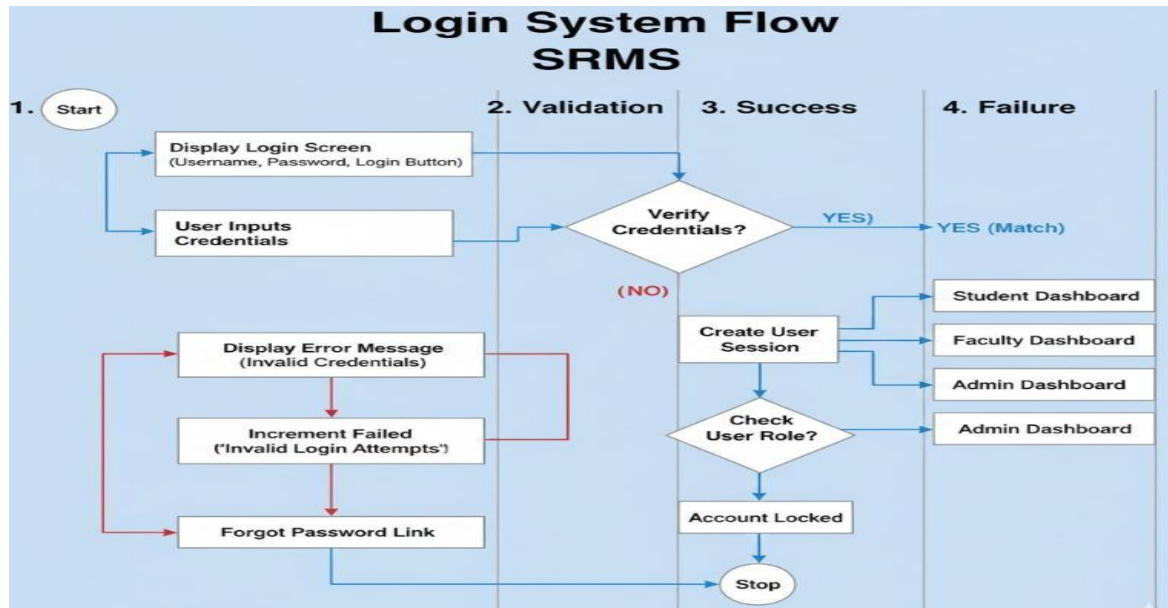
## **User Module:**

User has limited access with only essential options:

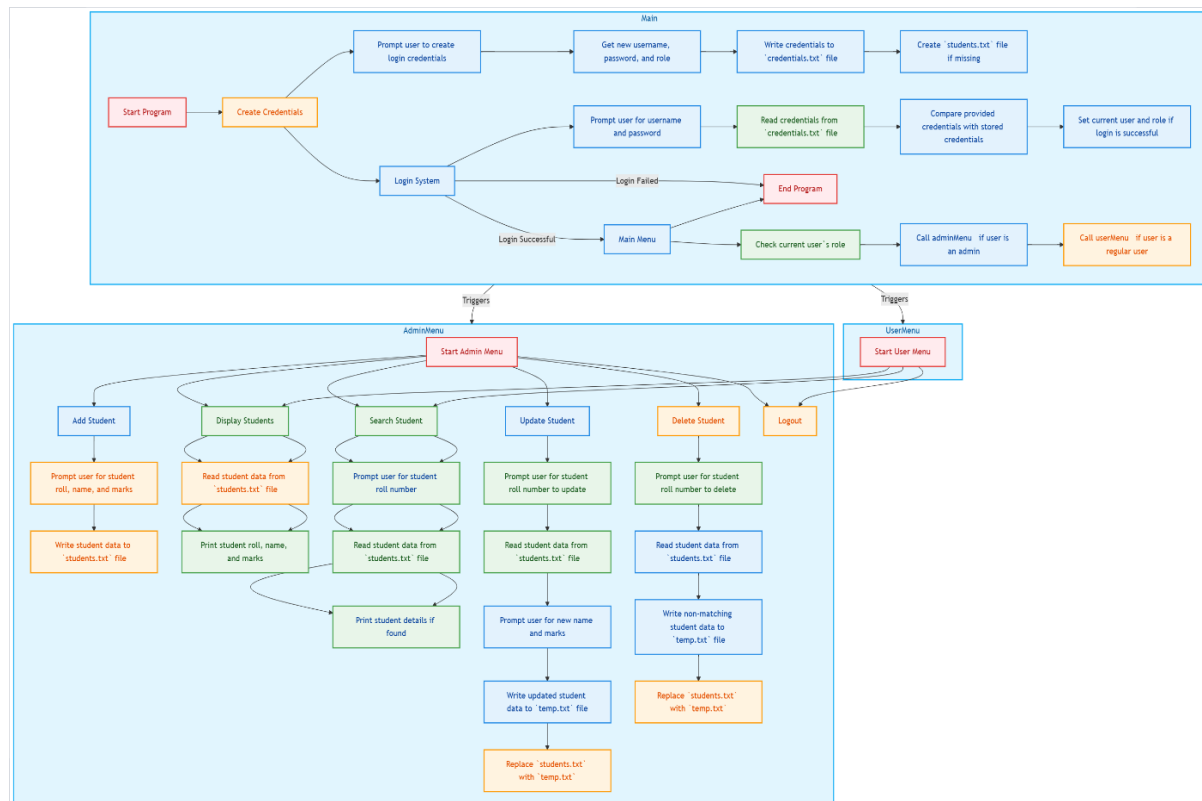
- Display all student records
- Search for specific student



## Flowchart



## Student Management Flow





## Code Execution

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define CREDENTIAL_FILE "credentials.txt"
#define STUDENT_FILE "students.txt"
struct student {
    int roll;
    char name[50];
    float marks;
};
char currentUser[20];
char currentRole[20];
void createCredentials();
int loginSystem();
void mainMenu();
void adminMenu();
void userMenu();
void addStudent();
void displayStudents();
void searchStudent();
void updateStudent();
void deleteStudent();
```



```
int main() {
    createCredentials();
    if (loginSystem()) {
        mainMenu();
    } else {
        printf("Login failed! Exiting...\n");
    }
    return 0;
}

void createCredentials() {
    char choice;
    char user[20], pass[20], role[10];
    printf("Do you want to create login credentials? (y/n): ");
    scanf(" %c", &choice);
    if (choice == 'y' || choice == 'Y') {
        printf("Enter new username: ");
        scanf("%s", user);
        printf("Enter new password: ");
        scanf("%s", pass);
        printf("Enter role (admin/user): ");
        scanf("%s", role);
        FILE *fp = fopen(CREDENTIAL_FILE, "w");
        if (!fp) {
```





```
        printf("Error creating credential file!\n");
        return;
    }
    fprintf(fp, "%s %s %s\n", user, pass, role);
    fclose(fp);

    printf("\nCredentials created successfully!\n");
}
FILE *f = fopen(STUDENT_FILE, "r");
if (!f) {
    f = fopen(STUDENT_FILE, "w");
    fclose(f);
} else {
    fclose(f);
}
}

int loginSystem() {
    char username[20], password[20];
    char fileUser[20], filePass[20], fileRole[20];
    printf("\n===== LOGIN SCREEN =====\n");
    printf("Enter username: ");
    scanf("%s", username);
    printf("Enter password: ");
```



```
scanf("%s", password);
FILE *fp = fopen(CREDENTIAL_FILE, "r");
if (!fp) {
    printf("Credential file missing!\n");
    return 0;
}
while (fscanf(fp, "%s %s %s", fileUser, filePass, fileRole) != EOF) {
    if (strcmp(username, fileUser) == 0 &&
        strcmp(password, filePass) == 0) {
        strcpy(currentUser, fileUser);
        strcpy(currentRole, fileRole);
        fclose(fp);
        return 1;
    }
}
fclose(fp);
return 0;
}

void mainMenu() {
    if (strcmp(currentRole, "admin") == 0)
        adminMenu();
    else
        userMenu();
}
```



```
}  
void adminMenu() {  
    int ch;  
    do {  
        printf("\n===== ADMIN MENU =====\n");  
        printf("1. Add Student\n");  
        printf("2. Display Students\n");  
        printf("3. Search Student\n");  
        printf("4. Update Student\n");  
        printf("5. Delete Student\n");  
        printf("6. Logout\n");  
        printf("Enter choice: ");  
        scanf("%d", &ch);  
        switch (ch) {  
            case 1: addStudent(); break;  
            case 2: displayStudents(); break;  
            case 3: searchStudent(); break;  
            case 4: updateStudent(); break;  
            case 5: deleteStudent(); break;  
            case 6: return;  
            default: printf("Invalid choice!\n");  
        }  
    } while (1);  
}
```



```
}  
void userMenu() {  
    int ch;  
    do {  
        printf("\n===== USER MENU =====\n");  
        printf("1. Display Students\n");  
        printf("2. Search Student\n");  
        printf("3. Logout\n");  
        printf("Enter choice: ");  
        scanf("%d", &ch);  
        switch (ch) {  
            case 1: displayStudents(); break;  
            case 2: searchStudent(); break;  
            case 3: return;  
            default: printf("Invalid choice!\n");  
        }  
    } while (1);  
}  
void addStudent() {  
    struct student st;  
    FILE *fp = fopen(STUDENT_FILE, "a");  
    printf("Enter roll number: ");  
    scanf("%d", &st.roll);
```



```
printf("Enter name: ");
scanf("%s", st.name);
printf("Enter marks: ");
scanf("%f", &st.marks);
fprintf(fp, "%d %s %.2f\n", st.roll, st.name, st.marks);
fclose(fp);
printf("Student added successfully!\n");
}

void displayStudents() {
    struct student st;
    FILE *fp = fopen(STUDENT_FILE, "r");
    printf("\nRoll\tName\tMarks\n");
    printf("-----\n");
    while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF)
    {
        printf("%d\t%s\t%.2f\n", st.roll, st.name, st.marks);
    }
    fclose(fp);
}

void searchStudent() {
    int roll;
    printf("Enter roll number to search: ");
    scanf("%d", &roll);
    struct student st;
```



```
FILE *fp = fopen(STUDENT_FILE, "r");
int found = 0;
while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF)
{
    if (st.roll == roll) {
        printf("\nFound: %d %s %.2f\n", st.roll, st.name, st.marks);
        found = 1;
        break;
    }
}

if (!found)
    printf("Student not found!\n");

fclose(fp);
}

void updateStudent() {
    int roll;
    printf("Enter roll number to update: ");
    scanf("%d", &roll);
    struct student st;
    FILE *fp = fopen(STUDENT_FILE, "r");
    FILE *tmp = fopen("temp.txt", "w");
    int found = 0;
```



```
while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF)
{
    if (st.roll == roll) {
        found = 1;
        printf("Enter new name: ");
        scanf("%s", st.name);
        printf("Enter new marks: ");
        scanf("%f", &st.marks);
    }
    fprintf(tmp, "%d %s %.2f\n", st.roll, st.name, st.marks);
}
fclose(fp);
fclose(tmp);
if (found) {
    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);
    printf("Record updated!\n");
} else {
    remove("temp.txt");
    printf("Roll number not found!\n");
}
}

void deleteStudent() {
    int roll;
```



```
printf("Enter roll number to delete: ");
scanf("%d", &roll);
struct student st;
FILE *fp = fopen(STUDENT_FILE, "r");
FILE *tmp = fopen("temp.txt", "w");
int found = 0;
while (fscanf(fp, "%d %s %f", &st.roll, st.name, &st.marks) != EOF)
{
    if (st.roll == roll) {
        found = 1;
        continue;
    }
    fprintf(tmp, "%d %s %.2f\n", st.roll, st.name, st.marks);
}

fclose(fp);
fclose(tmp);
if (found) {
    remove(STUDENT_FILE);
    rename("temp.txt", STUDENT_FILE);
    printf("Record deleted!\n");
} else {
    remove("temp.txt");
    printf("Roll number not found!\n");
}
```





```
}  
  
}
```

## OUTPUT

```
Do you want to create login credentials? (y/n): y  
Enter new username: HariVenakt  
Enter new password: AP24110011941  
Enter role (admin/user): admin
```

Credentials created successfully!

===== LOGIN SCREEN =====

```
Enter username: HariVenakt  
Enter password: AP24110011941
```

Login Successful!

===== ADMIN MENU =====

```
1. Add Student  
2. Display Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Logout  
Enter choice: 1  
Enter roll number: 2400040375  
Enter name: Revanth  
Enter marks: 97.5  
Student added successfully!
```

===== ADMIN MENU =====

```
1. Add Student  
2. Display Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Logout  
Enter choice: 2
```

| Roll | Name          | Marks |
|------|---------------|-------|
| 2    | AP4000040375  | 30.00 |
| 1    | AP24000040305 | 30.00 |
| 204  | AP24000040382 | 85.50 |

```
===== ADMIN MENU =====  
1. Add Student  
2. Display Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Logout  
Enter choice: 4  
Enter roll number to update: 204  
Enter new name: Sai  
Enter new marks: 95.3  
Record updated!
```

```
===== ADMIN MENU =====  
1. Add Student  
2. Display Students  
3. Search Student  
4. Update Student  
5. Delete Student  
6. Logout  
Enter choice: 5  
Enter roll number to delete: 204  
Record deleted!
```

## CONCLUSION

The development of the Student Management System marks a meaningful step toward transforming traditional record-keeping into a more accurate, secure, and automated process. By using the C programming language as the foundation, this project proves that even simple programming tools can create dependable real-world applications when used effectively. The system not only streamlines the handling of student information but also ensures that data remains protected through role-based access and permanent storage using text files.

Through features such as adding, updating, searching, and deleting records, the project demonstrates how structured data and modular programming can work together to produce a clean, organized, and user-friendly solution. The login-based access control further strengthens reliability by preventing unauthorized usage. In essence, this project achieves its purpose by delivering a lightweight, efficient, and practical system that can be easily extended or upgraded in the future. It stands as a strong example of how fundamental programming concepts can solve everyday administrative challenges with precision and clarity.